LAB REPORT 11

Strings in C++

Computer Programming · UET Nowshera · Electrical Engineering

1 Objective

The purpose of this lab was to explore how text data is represented and manipulated in C++. By the end of the session, the following points were to be understood practically:

- Recognising that a string is simply an ordered sequence of characters held together in memory
- Telling apart the older **C-String** style (character arrays) from the modern **C++ string class**
- Writing correct declarations and initialisations for char-array strings
- Calling standard library routines — `strcpy`, `strcat`, `strlen` — to carry out common string operations

2 Theoretical Background

C-Strings: Character Arrays with a Terminator

Before C++ introduced its own string class, text was handled the same way C does it: as a plain array of `char` elements with a special **null character** (`\0`) placed at the end. Every standard string routine relies on finding this sentinel byte to know where the text stops. A five-letter word like `"Hello"` therefore consumes six bytes of memory — one per letter plus the automatic terminator the compiler inserts.

Setting Up a C-String Variable

Writing `char myString[] = "Hello";` lets the compiler count the characters, size the array accordingly, and tack on the null byte — no manual counting required. The array may be declared larger than the content it currently holds; the string is still considered to end at the first `\0`, so the spare slots at the back are simply ignored.

The C++ String Class

The standard library's `string` type removes the constraints of fixed-size char arrays entirely. A string object expands or contracts on its own as text is added or removed, so the programmer never has to pre-calculate buffer sizes or worry about overflowing an array. This makes text processing noticeably more straightforward and far less error-prone.

Capturing Input That Contains Spaces

Ordinary `cin >>` treats any whitespace as a delimiter and stops there, which means a phrase like "Electrical Engineering" would only deliver "Electrical". For char arrays, `cin.get()` solves this by reading

everything up to the newline. For string objects, `getline(cin, variable)` does the same job and is generally the preferred choice.

3 Software Used

IDE	DEV-C++ IDE
COMPILER	C++ Compiler (bundled with DEV-C++)

4 Experimental Tasks

Task 1 — String Length Calculation

A string variable was set up and assigned a sample phrase. The `length()` member function was then called on it to retrieve the character count, which was printed alongside the original text.

C++ Code

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string phrase = "Computer Programming";
    cout << "String: " << phrase << endl;
    cout << "Length: " << phrase.length() << endl;
    return 0;
}
```

Task 2 — String Concatenation

Two string variables — one holding a first name and one holding a last name — were joined together with a space character inserted between them. The resulting full name was stored in a third variable and displayed.

● ● ● C++ Code

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string firstName = "Ali";
    string lastName  = "Hassan";
    string fullName  = firstName + " " + lastName;
    cout << "Full Name: " << fullName << endl;
    return 0;
}
```

Task 3 — C-String Functions: strcpy, strcat, strlen

Three classic C-string routines were put to work: `strcpy` duplicated one char array into another, `strcat` appended additional text to the copy, and `strlen` measured the final length. Each result was printed to verify the operation.

C++ Code

```
#include <iostream>
#include <cstring>
using namespace std;

int main() {
    char src[]  = "Hello";
    char dest[20];

    strcpy(dest, src);               // copy src into dest
    cout << "Copied: " << dest << endl;

    strcat(dest, " World");          // append to dest
    cout << "Concatenated: " << dest << endl;

    cout << "Length: " << strlen(dest) << endl;
    return 0;
}
```

Task 4 — Reading a Full Line with getline

`getline()` was used with a string object to capture user input that contained spaces — something the standard extraction operator cannot handle on its own. The captured text was echoed back to confirm the entire line was received correctly.

C++ Code

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string fullLine;
    cout << "Enter a sentence: ";
    getline(cin, fullLine);
    cout << "You entered: " << fullLine << endl;
    return 0;
}
```

5 Conclusion

Working through this lab made the distinction between C-strings and C++ string objects very concrete. C-strings demand careful attention to array sizing and null termination, while the string class handles all of that automatically, letting the programmer focus on what the text should do rather than how it is stored. The four tasks covered the full range of everyday string operations — measuring length, joining strings, using the C library functions, and capturing multi-word input — providing a practical toolkit for text-based programs.